

AERONEWSFRA / TECHNISCHE ZIELARCHITEKTUR

Professioneller Bauplan für AeroNewsFRA 2.0

Eine lokale Aviation-Data-Plattform für Frankfurt: echte ADS-B-Daten, geografische Live-Karte, geprüfter Newsroom, KI-gestützte Produktion und skalierbare Einnahmequellen.

Version 1.0 · Planungsstand August 2026

Arbeitsdokument für Entwicklung, Betrieb und spätere Automatisierung

Dieses Dokument beschreibt nicht nur einzelne Webseitenfunktionen, sondern das vollständige technische Zielsystem. Die bestehende statische GitHub-Pages-Seite bleibt die öffentliche Oberfläche. Echtzeitdaten, Historie, KI und redaktionelle Prozesse werden über ein separates, abgesichertes Backend ergänzt.

Leitentscheidung: AeroNewsFRA wird keine Kopie von Flightradar24. Der Mehrwert ist die lokale, nachvollziehbare Einordnung: Was ist rund um Frankfurt interessant, warum ist es interessant und welche Daten belegen es?

Kap.	Bereich	Ziel
1	Ausgangslage	Bestand, Grenzen und Zielbild
2	Produkt und Seiten	Funktionen der gesamten öffentlichen Seite
3	Karte und Geodaten	Echte Live-Karte statt SVG-Simulation
4	ADS-B und Datenplattform	Empfang, Speicherung, API und Historie
5	KI und Redaktion	Automatisierung mit Sicherheitsgrenzen
6	Technik und Betrieb	Hosting, Security, Tests und Monitoring
7	Roadmap	Realisierbare Phasen und Abnahme
8	Entwicklerprofil	Fähigkeiten und Arbeitsweise

Technische Grundsätze

Fakten vor KI

Zahlen kommen aus validierten Daten. KI formuliert, erfindet aber keine Fakten.

Live plus Historie

Aktuelle Positionen und dauerhaft auswertbare Bewegungen werden getrennt behandelt.

Review vor Veröffentlichung

Automatisierung bereitet vor. Sicherheits- und Nachrichteninhalte benötigen Freigabe.

Abgrenzung

Öffentlich werden nur Daten angezeigt, deren Lizenz, Genauigkeit und Zweck geklärt sind. Der Standort der Empfangsanlage wird nicht präzise veröffentlicht. Die Seite ist unabhängig und kein offizieller Kanal von Fraport oder Frankfurt Airport.

Vorhandener Stand

- GitHub-Repository **Webseite-aeronewsfra** mit statischem HTML, CSS und JavaScript.
- GitHub Pages als funktionierende öffentliche Hosting-Plattform.
- Nachrichtenquelle `content/news.xlsx`, erzeugt als `data/news.json`.
- GitHub Actions für tägliche Recherchekandidaten und Seiten-Deployment.
- Eigene News-, Movements-, Dashboard-, Spotting-, Quellen- und Kartenseiten.
- SVG-Karte mit Zoom, Messung, Standort und optionaler OpenSky-Abfrage.

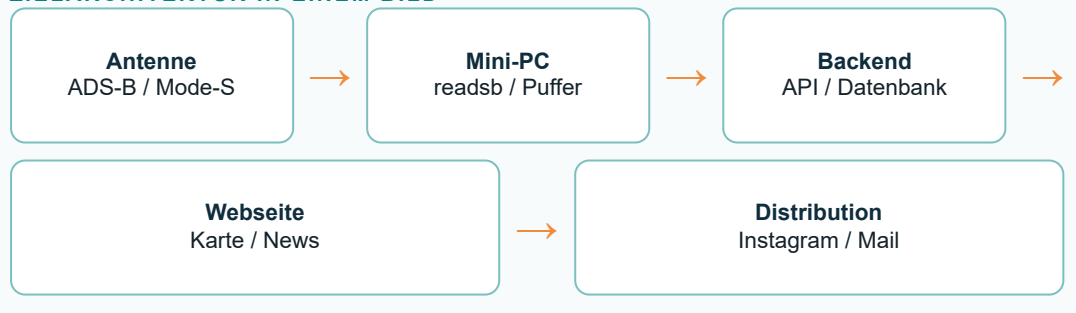
Aktuelle Grenzen

- Die Karte ist schematisch und nutzt keine echten Kartenkacheln.
- OpenSky wird direkt vom Browser aufgerufen; dadurch entstehen Rate-Limits und keine zentrale Datenqualität.
- Keine eigene dauerhafte ADS-B-Datenannahme.
- Keine historische Positionsdatenbank und keine Replay-Funktion.
- Review-Kandidaten werden nicht automatisch zu öffentlichen Artikeln.
- Kein Backend für Benutzer, Newsletter, Push oder Premium-Funktionen.

Zielbild

- Echte geografische Karte mit klaren Daten- und Lizenzhinweisen.
- Eigener Receiver-Feed mit Status und Datenqualität.
- Backend mit Cache, API, Historie und Ereigniserkennung.
- Automatischer Content-Entwurf für Webseite und Instagram.
- Ein täglicher Prüfpunkt statt laufender manueller Recherche.
- Erweiterbar für Newsletter, Affiliate und Premium-Alarme.

ZIELARCHITEKTUR IN EINEM BILD



Erfolgskriterium: Ein Besucher versteht innerhalb von zehn Sekunden, was heute in Frankfurt passiert. Ein Entwickler kann jede angezeigte Zahl bis zu einer Quelle und einem Empfangszeitpunkt zurückverfolgen.

Seitenkonzept für das gesamte Portal

Öffentliche Oberfläche

Startseite

Die Startseite wird zur tagesaktuellen Lageübersicht. Oben stehen Datenalter, Receiver-Status, Zahl der interessanten Bewegungen und die wichtigste Meldung. Darunter folgen „Heute rund um FRA“, Wetter- und Spotting-Hinweise sowie Links zu Karte und Newsroom.

Newsroom

Artikel erhalten eigene URLs, Kategorien, Quellen, Zeitstempel, verwandte Meldungen und strukturierte SEO-Daten. Filter, Suche, Archiv, RSS und Social-Share machen aus der Liste einen dauerhaft auffindbaren Newsroom.

Special Movements

Diese Seite nutzt die Ereigniserkennung des Backends. Sie zeigt seltene Typen, Erstbesuche, Fracht, Regierungsflüge, Umleitungen, Go-arounds und historische Flugzeuge. Jeder Eintrag trägt einen Status wie „beobachtet“, „geprüft“ oder „Quelle bestätigt“.

Dashboard

Öffentliche Kennzahlen: Meldungen, Bewegungen, Flugzeugtypen, Airlines, Datenalter, Stations-Uptime, Empfangsreichweite und Monatsverlauf. Der exakte Hausstandort bleibt verborgen.

Spotting

Interaktive Punkte mit Blickrichtung, Wetter, Sonnenstand, erwarteter Flugrichtung, rechtlichen Hinweisen und eigenen Bildern nach Moderation.

Quellen

Quellenstatus, Methodik, Datenherkunft, Lizenzhinweise, Korrekturhistorie und Kennzeichnung eigener ADS-B-Daten.

Kontakt

Serverloses Formular mit Spam-Schutz für Korrekturen, Bildangebote und Faktenhinweise. Keine persönliche Wohnadresse öffentlich exponieren.

Technische Portal-Funktionen

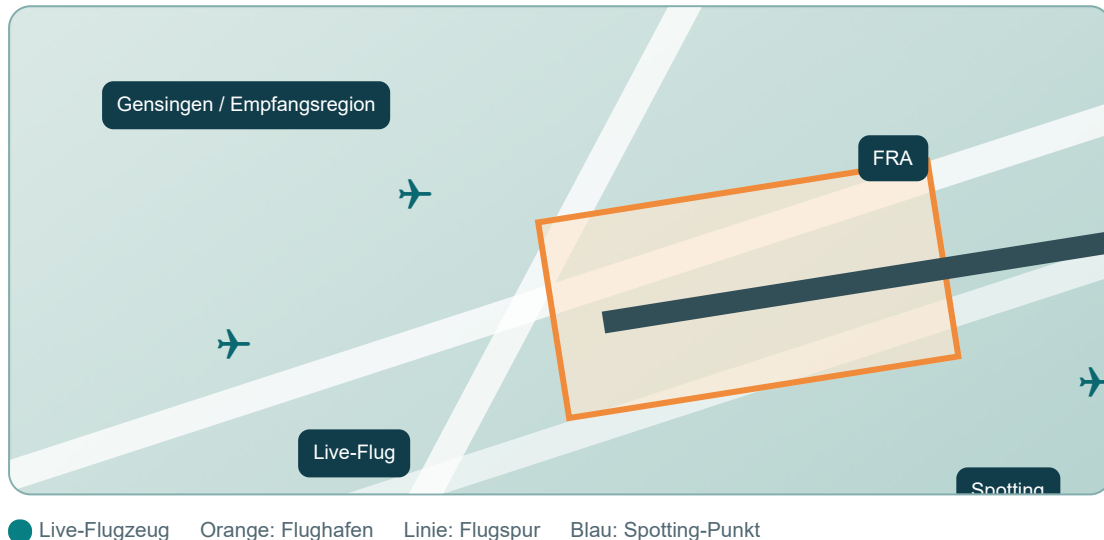
- Responsive Mobile-first-Layout
- Barrierefreiheit nach WCAG 2.2 AA
- Breadcrumbs und Canonical-URLs
- OpenGraph- und Twitter-Karten
- JSON-LD für NewsArticle und Dataset
- RSS/Atom und Sitemap
- Offline-fähige PWA-Shell
- Installierbares Web-App-Icon
- Dark- und Light-Modus
- Mehrsprachigkeit Deutsch/Englisch
- Seiten- und Datenstatus
- Fehler- und Leerzustände

Designregel: Nicht jede technische Funktion muss auf die Startseite. Live-Karte, Datenexplorer und historische Analyse gehören in eigene Werkzeuge; die Startseite bleibt schnell und redaktionell verständlich.

Die Karte als echtes technisches Kernprodukt

MapLibre oder
Leaflet

Die bestehende SVG-Karte soll durch eine echte Webkarte ersetzt werden. Für freie, flexible Darstellung ist MapLibre GL JS oder Leaflet geeignet. Die Kartenbasis muss einen erlaubten Tile-Anbieter und sichtbare Attribution verwenden.



Layer-System

Layer	Inhalt	Quelle	Verhalten
Basiskarte	Orte, Straßen, Gewässer	lizenzierter Tile-Anbieter	Zoom, Attribution, Cache
Airport	Runways, Terminals, Cargo	eigene GeoJSON-Daten	statisch, korrekt versioniert
Live-Flüge	Position, Track, Höhe, Callsign	eigener Receiver/API	Polling oder SSE, Clustering
Historie	Spuren und Replay	eigene Datenbank	Zeitregler, Filter
Wetter	Wind, Sicht, Wolken	Open-Meteo oder Anbieter	manuell oder zeitgesteuert
Spotting	öffentliche Punkte, Blickrichtung	redaktionell geprüft	Detailpanel, keine Privatstandorte

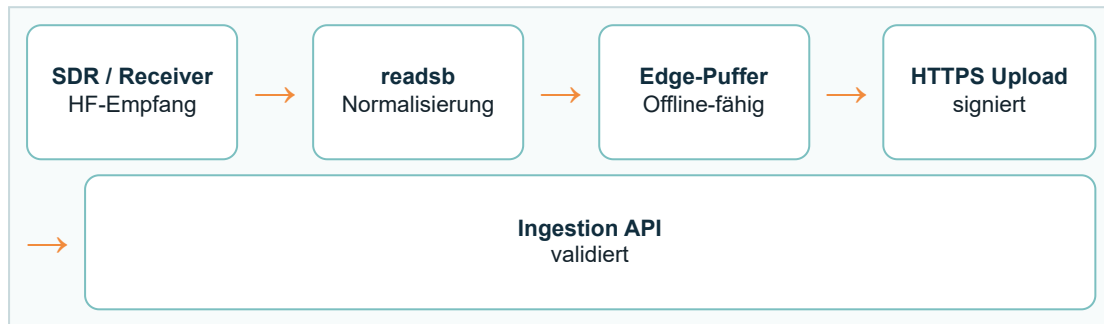
Bedienung und Datenfluss

- Flugzeuge als Marker mit Richtungspfeil und farblicher Höhenlogik.
- Klick öffnet Detailkarte mit Datenalter und Vertrauensstatus.
- Automatische Aktualisierung alle 10–30 Sekunden, aber mit Server-Cache.
- Bei veralteten Daten werden Marker ausgegraut, nicht stillschweigend weiterbewegt.
- Zu viele Punkte werden geclustert; die Karte bleibt auf Mobilgeräten flüssig.
- Ein Zeitregler wechselt zwischen Live-Modus und historischen Spuren.
- Geteilte Karten-URLs enthalten Ansicht, Zoom, Filter und Zeitraum.

Risiko: OpenStreetMap-Kacheln dürfen nicht beliebig belastet werden. Tile-Provider, Attribution, Caching und Nutzungsbedingungen werden vor dem Live-Betrieb verbindlich festgelegt.

Empfangsstation

Bei dir steht ein ADS-B-Empfänger mit geeigneter Antenne und Mini-PC. Die Software empfängt Mode-S/ADS-B, normalisiert Nachrichten und überträgt ausschließlich die benötigten Daten. Der Empfänger wird niemals direkt ins Internet geöffnet.



Datenmodell

Entität	Wichtige Felder	Aufbewahrung
Aircraft	icao24, registration, type, airline	dauerhaft, sofern Datenquelle erlaubt
Position	lat, lon, altitude, track, speed, received_at	Rohdaten kurz; Aggregate länger
Flight	callsign, departure, arrival, first_seen, last_seen	historisch nach Policy
Event	event_type, score, evidence, review_status	redaktionell dauerhaft
Source	provider, URL, checked_at, license	Artikel-Lebensdauer
Station	health, uptime, range_bucket, last_seen	Aggregate öffentlich

API-Endpunkte

```

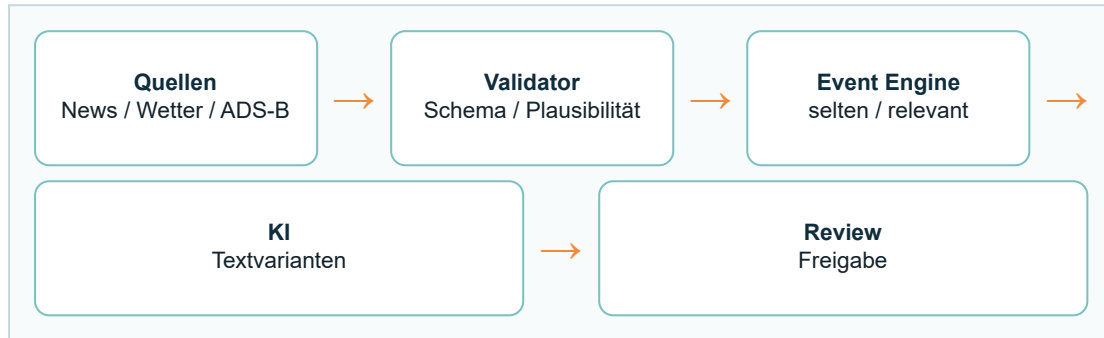
GET /api/v1/health
GET /api/v1/aircraft/live?bbox=...
GET /api/v1/aircraft/{icao24}
GET /api/v1/tracks/{flight_id}
GET /api/v1/events?from=...&to=...
GET /api/v1/stats/daily
GET /api/v1/station/status
POST /api/v1/receiver/heartbeat
POST /api/v1/receiver/positions
  
```

Datenqualität

- Jede Position erhält Empfangszeitpunkt und Datenalter.
- Unplausible Koordinaten, Sprünge und unmögliche Geschwindigkeiten werden verworfen.
- Confidence Score kombiniert Signalalter, Feldvollständigkeit und Plausibilität.
- Dashboard zeigt „aktuell“, „verzögert“ oder „offline“ statt falscher Präzision.
- Rohdaten werden begrenzt gespeichert; für die Webseite reichen Aggregate und ausgewählte Spuren.

Abnahme: Wird der Mini-PC vom Netz getrennt, zeigt die Webseite innerhalb eines definierten Zeitfensters „Empfang offline“. Nach Wiederkehr werden keine doppelten oder zeitlich falsch eingeordneten Bewegungen erzeugt.

Die KI ist ein Produktionswerkzeug, kein unabhängiger Redakteur. Sie erhält ein strukturiertes Faktenpaket und darf nur daraus formulieren. Jede Veröffentlichung bleibt durch Status und Freigabe kontrollierbar.



Automatisierte Ereignisse

Bewegungslogik

Erstbesuch, seltener Flugzeugtyp, ungewöhnliche Airline, Umleitung, Go-around, Holding, Fracht- oder Regierungsflug.

Redaktionelle Logik

FRA-Bezug, Neuigkeitswert, Quellenqualität, Sichtbarkeit, Duplikate und potenzielles Fehlinterpretationsrisiko.

Statusmodell

Status	Bedeutung	Öffentlich?
detected	technisch erkannt	nein
candidate	als interessant markiert	nein
review	Quellen- und Faktenprüfung	nein
verified	redaktionell bestätigt	noch nicht
scheduled	für Veröffentlichung vorgesehen	nein
published	öffentlich	ja
corrected / archived	Korrektur oder Archiv	abhängig

KI-Ausgaben

- Artikelentwurf und Kurzfassung
- Instagram-Caption und Story-Variante
- Reel- oder Short-Skript
- SEO-Titel, Meta-Description und Alt-Text
- deutsche und englische Fassung
- Bildbriefing, aber keine ungeprüften Fremdbilder

Verboten: erfundene Flugnummern, Quellen, Ursachen, Sicherheitsbewertungen oder exakte private Positionsangaben. Unfall- und Sicherheitsmeldungen werden nie ohne ausdrückliche menschliche Prüfung veröffentlicht.

Hosting-Aufteilung

Komponente	Empfehlung	Begründung
Frontend	GitHub Pages	kostenarm, schnell, statisch, CDN
API	Cloudflare Worker, VPS oder vergleichbarer Dienst	Cache, Rate-Limits und sichere Endpunkte
Datenbank	PostgreSQL; SQLite für lokalen Puffer	Historie, Filter, Indizes, zuverlässige Backups
Receiver	Mini-PC bei der Antenne	geringer Verbrauch, lokale Ausfallsicherheit
Bilder	Objektspeicher mit HTTPS	Meta API benötigt öffentliche Bild-URL
Automation	GitHub Actions plus Backend-Jobs	Recherche, Build, Monitoring und Reports

Sicherheitsstandard

- Secrets ausschließlich über Secret Store
- keine Meta- oder API-Schlüssel im Repository
- HTTPS und signierte Receiver-Uploads
- API-Key pro Station mit Rotation
- strenge CORS-Regeln
- Content-Security-Policy
- Rate-Limits und Bot-Schutz
- exakten Hausstandort nicht veröffentlichen
- EXIF-Daten aus Bildern entfernen
- Logs ohne private Daten
- Quellen- und Bildrechte dokumentieren
- öffentliche Datenanbieterbedingungen prüfen
- Backups und getesteter Restore
- schriftliche Zustimmung für Antennenbetrieb

Testpyramide

Unit

Koordinaten, Distanz, Zeit, Filter, Plausibilität und Statusübergänge.

Integration

Receiver, API, Datenbank, OpenSky-Fallback, Wetter und Bildspeicher.

Browser

Alle Seiten, Karteninteraktion, mobile Bedienung, Leer- und Offline-Zustände.

Monitoring

Überwacht werden Seitenstatus, API-Latenz, JavaScript-Fehler, Receiver-Uptime, Datenalter, fehlgeschlagene Jobs, Build-Status und Bild-Uploads. Bei einem Fehler gibt es eine sichtbare Statusanzeige und optional eine Benachrichtigung.

Deployment-Regel: Jede Änderung läuft zuerst durch Build, Validierung, Tests und Preview. Danach erfolgt der Live-Deploy. Ein Rollback auf den letzten funktionierenden Stand muss mit einem Klick oder einem eindeutigen Commit möglich sein.

Roadmap und konkrete Arbeitspakete

Priorisiert

Phase	Arbeitspakete	Abnahme
0 Stabilität	Repository sauber anbinden; Build-Workflow reparieren; Review- und Published-Daten trennen; Tests und Statusseite ergänzen.	Neue freigegebene Meldung erscheint nach einem erfolgreichen Deploy zuverlässig öffentlich.
1 Echte Karte	MapLibre/Leaflet; Kartenprovider; Attribution; Airport-GeoJSON; Marker, Layer, Clustering und Fehlerzustände.	Die Karte zeigt echte Orte, Flughafen und belastbare Koordinaten auf Desktop und Mobilgerät.
2 Live-API	Backend, Cache, Health-Endpunkt, OpenSky-Proxy als Übergang, Datenalter und automatische Aktualisierung.	Rate-Limits werden abgefangen; Offline- und verzögerte Daten werden korrekt angezeigt.
3 Eigener Receiver	SDR, readsb, Antenne, lokaler Puffer, signierter Upload, Stations-Dashboard und Reichweiten-Aggregate.	Eigene Testbewegung erscheint in der API; Ausfall wird korrekt erkannt.
4 Historie	Flight Tracks, Tagesstatistik, Replay, Zeitfilter, Flugzeug- und Airline-Suche.	Ein vergangener Zeitraum kann reproduzierbar auf Karte und Dashboard dargestellt werden.
5 Automation	Event Engine, Quellen-Checks, KI-Entwürfe, Bildkarten, Instagram- und Newsletter-Draft.	Ein geprüfter Datensatz erzeugt ohne Copy-Paste mehrere kontrollierbare Entwürfe.
6 Produkt	SEO-Artikel, Affiliate-Flächen, Newsletter, Premium-Alarme, Analytics und Consent.	Messbare Anmeldung oder Klickaktion ohne unzulässige Werbung oder Datenschutzfehler.

Definition of Done für jede Funktion

1. Nutzen und Datenquelle sind dokumentiert.
2. Desktop- und Mobilansicht funktionieren.
3. Leere, verzögerte und fehlerhafte Daten sind gestaltet.
4. Automatisierte Tests decken den Kernfall ab.
5. Keine Secrets, private Standorte oder unklaren Lizenzen enthalten.
6. Preview und Live-Deployment wurden geprüft.
7. Rollback und Monitoring sind vorhanden.

Erstes realistisches MVP

Das erste wertvolle Release besteht aus echter Basiskarte, eigenem API-Cache, Live-Fluglayer, Statusanzeige, Flugzeugdetailpanel und sauberem News-Update. Historische Replay-Funktion und Monetarisierung folgen erst, wenn die Datenbasis zuverlässig läuft.

Keine Big-Bang-Entwicklung: Erst die Datenkette beweisen, dann zusätzliche Layer, KI und Einnahmefunktionen hinzufügen. Jede Phase muss einzeln nutzbar und rücksetzbar sein.

Wer dieses System entwickeln sollte

Anforderungsprofil

Gesucht ist kein reiner Webdesigner und kein reiner KI-Prompt-Entwickler. Die verantwortliche Person muss Produkt, Daten, Karten, Backend und redaktionelle Sicherheit zusammen denken können.

Fachliche Kernfähigkeiten

- HTML, CSS, JavaScript, TypeScript
- responsive und barrierefreie Webentwicklung
- Python/FastAPI oder Node.js
- REST, SSE/WebSockets und API-Design
- SQLite/PostgreSQL und Datenmodellierung
- GeoJSON, WGS84, Web-Mercator und GIS
- MapLibre oder Leaflet
- ADS-B, Mode-S und Flugphasen
- Raspberry Pi/Linux/Netzwerk
- Git und GitHub Actions
- Cloudflare, VPS oder vergleichbares Hosting
- Monitoring, Logs und Backups
- OAuth, Secrets und Web-Sicherheit
- SEO, strukturierte Daten und Performance
- KI-Integration mit validierten Eingaben
- Quellen-, Bild- und Datenlizenzen

Arbeitsweise

1. **Bestand lesen:** Vor jeder Änderung Repository, Workflow, Datenmodell und Live-Seite prüfen.
2. **Klein schneiden:** Ein Feature pro Branch oder Pull Request; keine unkontrollierte Komplettschreibung.
3. **Daten zuerst:** Jede Darstellung braucht Quelle, Zeitstempel, Schema und Fehlerzustand.
4. **Sicher entwickeln:** Keine Zugangsdaten im Chat, Code oder öffentlichen Repository.
5. **Redaktion schützen:** KI liefert Vorschläge; riskante Inhalte bleiben manuell.
6. **Testen:** Unit, Integration, Browser, Mobil, Offline und Rate-Limit-Fälle prüfen.
7. **Dokumentieren:** Architektur, API, Datenrechte, Betriebsanleitung und Rollback aktuell halten.
8. **Rückwärtskompatibel bleiben:** Bestehende Seiten funktionieren während des Ausbaus weiter.

Persönliche Eigenschaften

Präzision

Keine erfundenen Werte, keine unklaren Quellen und kein Verschweigen von Datenlücken.

Pragmatismus

Die kleinste sichere Lösung zuerst; teure Infrastruktur erst bei nachgewiesenem Bedarf.

Verantwortung

Datenschutz, Lizenzen, Sicherheit und falsche Sicherheitsmeldungen werden ernst genommen.

Produktdenken

Jede technische Besonderheit muss einen Besucher-, Redaktions- oder Einnahmenvorteil haben.

Abschlussprüfung

Vor jedem Release beantwortet der Entwickler: Ist die Funktion korrekt? Ist sie verständlich? Ist sie mobil bedienbar? Was passiert bei fehlenden Daten? Darf die Quelle so verwendet werden? Wie kann man den Release zurücknehmen?

Vor dem ersten echten Live-Datenbetrieb

- Schriftliche Zustimmung der Hauseigentümer für Antenne und Geräte.
- Standort in der öffentlichen Darstellung grob verschleiert.
- Receiver, Stromversorgung, Überspannungs- und Netzwerkschutz geprüft.
- Datenanbieter, Kartenkacheln und Bildquellen auf Nutzungsrechte geprüft.
- Backend nicht über das Heimnetz direkt öffentlich exponiert.
- API-Keys und Meta-Zugangsdaten in Secrets hinterlegt.
- Backups, Restore-Test und Rollback dokumentiert.

Vor jeder Veröffentlichung

- Fakten stammen aus sichtbaren, dokumentierten Quellen.
- ADS-B-Beobachtung und redaktionelle Bestätigung sind getrennt markiert.
- Datum, Zeitzone und Datenalter stimmen.
- Keine privaten oder sicherheitskritischen Details.
- Bildrechte und Alt-Text sind geklärt.
- Instagram- und Webseitenfassung widersprechen sich nicht.

Messbare Ziele für die ersten 90 Tage

KPI	Messung	Richtung
Datenverfügbarkeit	API und Receiver-Uptime	stabil, Ausfälle sichtbar
Aktualität	Median des Datenalters	klar ausgewiesen
Redaktion	Zeit von Kandidat zu geprüftem Entwurf	immer weniger manuelle Schritte
Qualität	Korrekturen und Fehlalarme	niedrig, nachvollziehbar
Produkt	wiederkehrende Besucher und Newsletter	monatlich steigend
Einnahmen	Affiliate-Klicks, Produktverkäufe, Unterstützer	erst nach Reichweitenbasis

Endzustand: AeroNewsFRA empfängt regionale Flugdaten, macht daraus nachvollziehbare Informationen, stellt sie als echte Karte und News dar und verteilt geprüfte Inhalte mit möglichst wenig täglichem Aufwand. Die Technik bleibt transparent, wartbar und sicher.

Priorisierte erste drei Tickets

1. **Deployment reparieren:** Tägliche freigegebene Daten zuverlässig auf GitHub Pages veröffentlichen.
2. **Echte Karte vorbereiten:** Kartenbibliothek, Provider, GeoJSON und Datenzugriff entkoppeln.
3. **Receiver-Schnittstelle definieren:** Datenformat, Health-Heartbeat und Upload-Sicherheit festlegen, bevor Hardware gekauft wird.

Dieses Dokument ist eine technische Planungsgrundlage. Vor produktivem Betrieb sind aktuelle Anbieterbedingungen, Datenschutz, Steuerfragen sowie lokale und luftfahrtbezogene Vorgaben gesondert zu prüfen.